

Experiment No.	1
Experiment Title	Exploratory Data Analysis, Preprocessing and Feature Selection on Tabular and Image Datasets
Student Name	S.R.Aadhithya
Register Number	3122247001001
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/26

1 Objective

The goal of this experiment is to walk through the early stages of a machine learning pipeline - exploration, cleaning, encoding, scaling and feature selection - on four fairly different datasets: Iris, a diabetes-risk table, a loan-approval table, and two handwritten character image sets (MNIST digits and a custom `english.csv` character set). Rather than jumping straight into modelling, the idea is to first understand what each dataset actually looks like, decide what preprocessing it needs, and pick out the features that carry the most signal, before finally training a baseline classifier on the two image datasets to see how far the extracted features actually get us.

2 Problem Statement

Four separate problems are handled in this notebook:

- **Iris** - a 3-class classification problem: predict the species of a flower (Setosa, Versicolor, Virginica) from four measurements (sepal/petal length and width).
- **Diabetes Prediction** - a binary classification problem: predict whether a patient has diabetes from demographic and health attributes (age, BMI, HbA1c, glucose level, etc.).
- **Loan Approval** - a binary classification problem: predict whether a loan application is approved or rejected from applicant income, assets, and credit score.
- **Handwritten Recognition** - a multi-class classification problem across two sources: MNIST (10 digit classes, 28×28 grayscale images) and a smaller custom set of 62 character classes (digits, uppercase and lowercase letters).

For the tabular datasets, the pipeline stops at preprocessing and feature selection (train/test split ready for a future modelling experiment). For the two image datasets, a Random Forest classifier is trained end-to-end so the effect of the preprocessing choices (PCA-based dimensionality reduction, in particular) can actually be measured.

3 Dataset Description

1. Iris Dataset

Dataset Name	Iris
Dataset Source	Iris.csv
Number of Samples	150
Number of Features	4 numeric (Id dropped)
Number of Classes	3
Missing Values	0
Train-Test Split	80 / 20

2. Diabetes Prediction

Dataset Name	Diabetes Prediction
Dataset Source	diabetes_prediction_dataset.csv
Number of Samples	100,000
Number of Features	8
Number of Classes	2
Missing Values	0 (3,854 duplicate rows)
Train-Test Split	80 / 20, stratified

3. Loan Approval

Dataset Name	Loan Approval
Dataset Source	loan_approval_dataset.csv
Number of Samples	4,269
Number of Features	11 (loan_id dropped)
Number of Classes	2
Missing Values	0
Train-Test Split	80 / 20, stratified

4. MNIST Digits

Dataset Name	MNIST Digits
Dataset Source	IDX ubyte files
Number of Samples	60,000 train / 10,000 test
Number of Features	784 ($28 \times px$)
Number of Classes	10
Missing Values	0
Train-Test Split	45,000 / 15,000 / 10,000

5. Character Recognition

Dataset Name	Character Recognition
Dataset Source	english.csv + images
Number of Samples	3,410
Number of Features	784 ($28 \times px, resized$)
Number of Classes	62
Missing Values	0
Train-Test Split	2,046 / 682 / 682

4 Exploratory Data Analysis

4.1 Iris

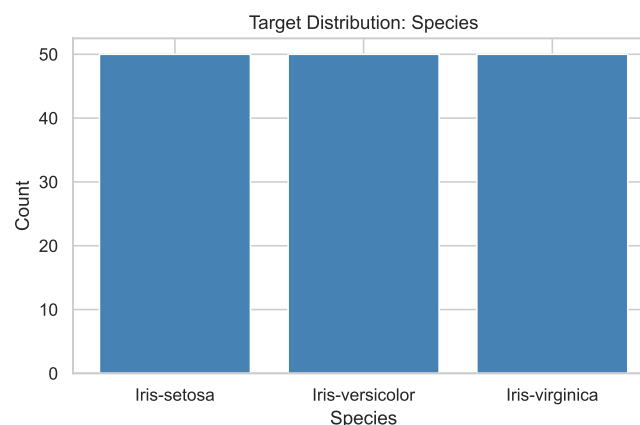


Figure 1: Iris - class distribution of the target (Species)

Inference: The three species are represented in exactly equal numbers, 50 samples each. There's no class imbalance to worry about here, which is one

less thing to correct for before modelling. It also means accuracy alone will be a fair enough metric later, without needing to lean on F1 or ROC-AUC to compensate for a skewed target.

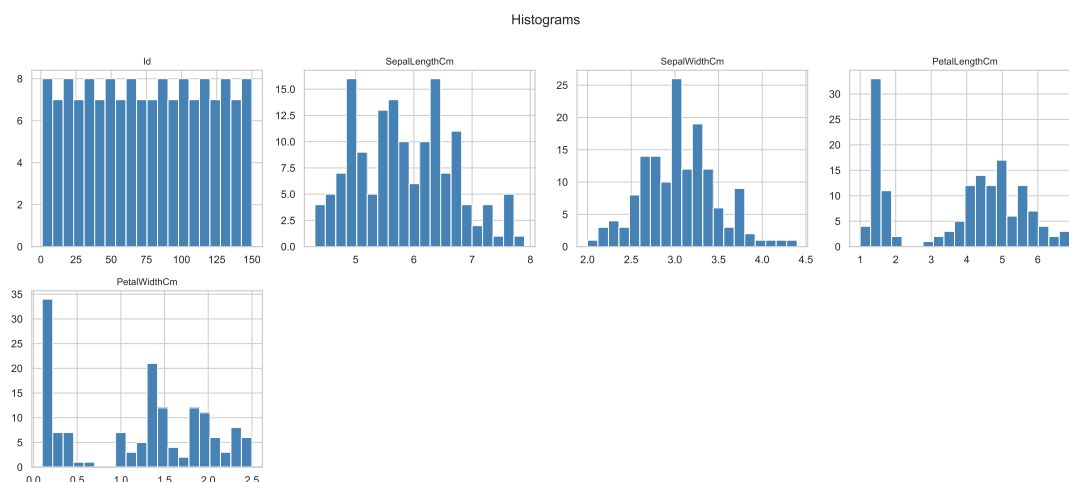


Figure 2: Iris - feature histograms

Inference: Petal length and petal width both show a visible gap in their distributions, roughly splitting the data into a small cluster of low values and a larger spread of higher ones. Sepal length and sepal width look closer to a single unimodal bump each, with sepal width the most symmetric of the four. This early on it already hints that the petal measurements will do most of the work in telling the species apart.

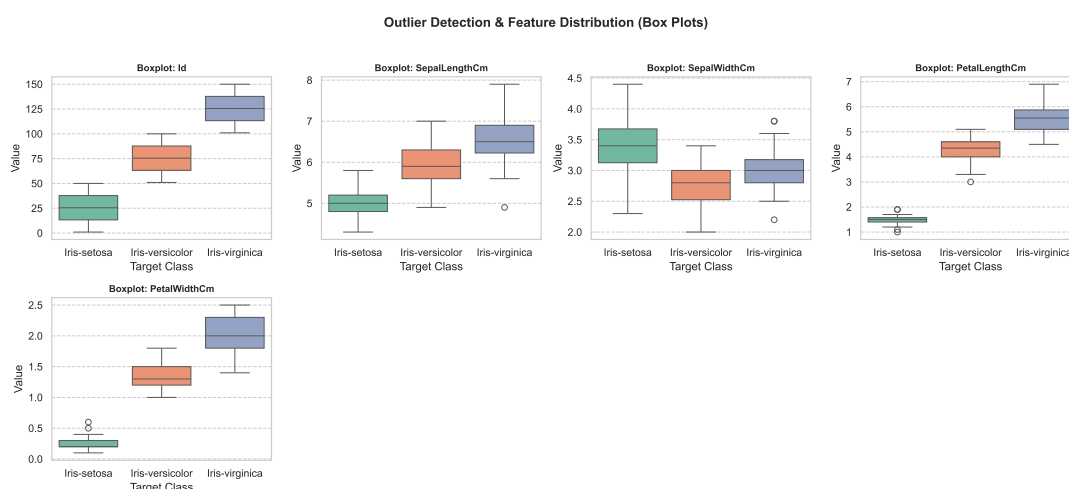


Figure 3: Iris - feature distributions grouped by species

Inference: Once split by species, petal length and petal width separate almost perfectly - Setosa sits in its own tight band well below the other two species, with very little overlap. Sepal width is the opposite case: the three

species' boxes overlap heavily, so on its own it wouldn't separate the classes well. Sepal length falls somewhere in between, useful but noisier than the petal features.

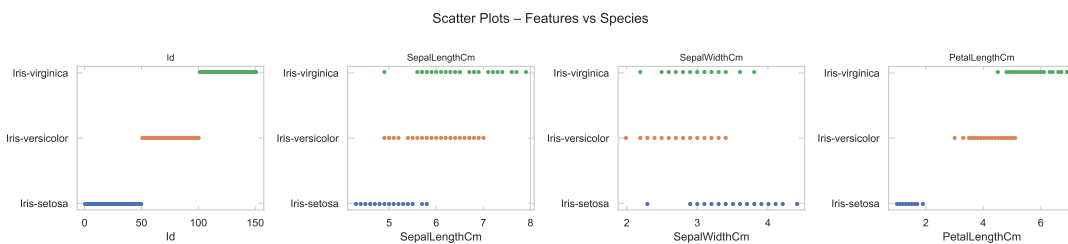


Figure 4: Iris - feature values plotted against species

Inference: These scatter plots back up what the boxplots already showed: Setosa points cluster tightly and away from the other two species on the petal features, while Versicolor and Virginica sit closer together with some overlap, particularly on sepal measurements. Separating Versicolor from Virginica is clearly the harder part of this classification task.

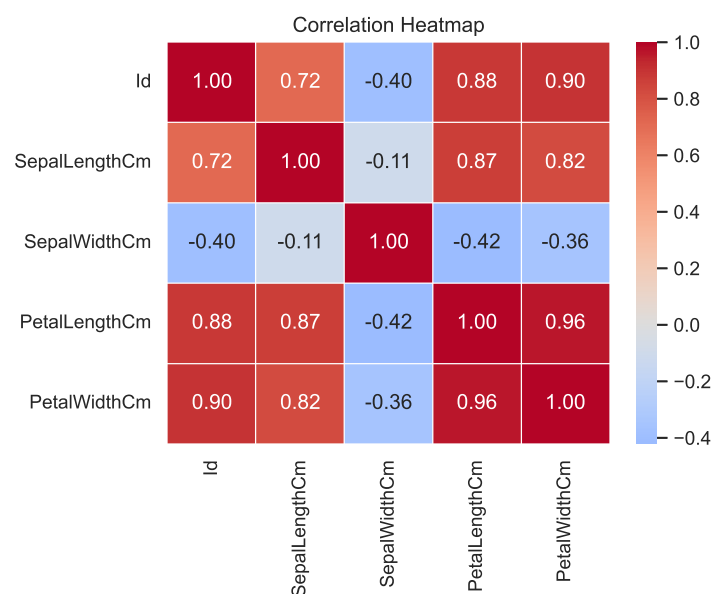


Figure 5: Iris - correlation heatmap of numeric features

Inference: Petal length and petal width are very strongly correlated (around 0.96), which isn't surprising since larger petals tend to be both longer and wider. Sepal length also correlates moderately with both petal measurements. Sepal width stands out as the odd one out, with a weak negative correlation to the rest, which lines up with it being the least useful feature in the boxplots above.

4.2 Diabetes Prediction

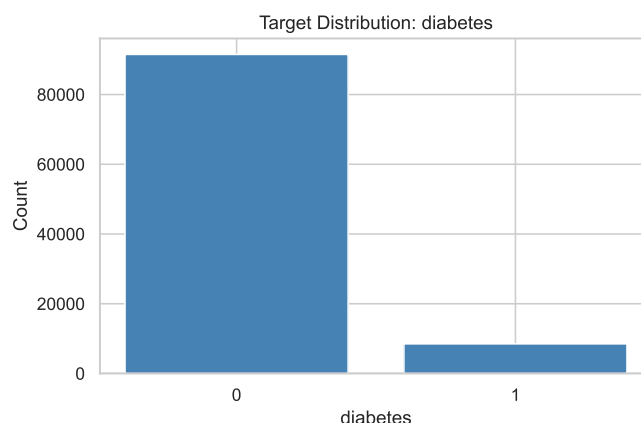


Figure 6: Diabetes - class distribution of the target

Inference: This dataset is heavily imbalanced - only about 8.5% of the 100,000 records are positive for diabetes. A model that simply predicts "no diabetes" for everyone would already score above 90% accuracy, so accuracy alone would be a misleading metric here. This is exactly why the notebook flags F1-score and ROC-AUC as the metrics to actually pay attention to once a model is trained.

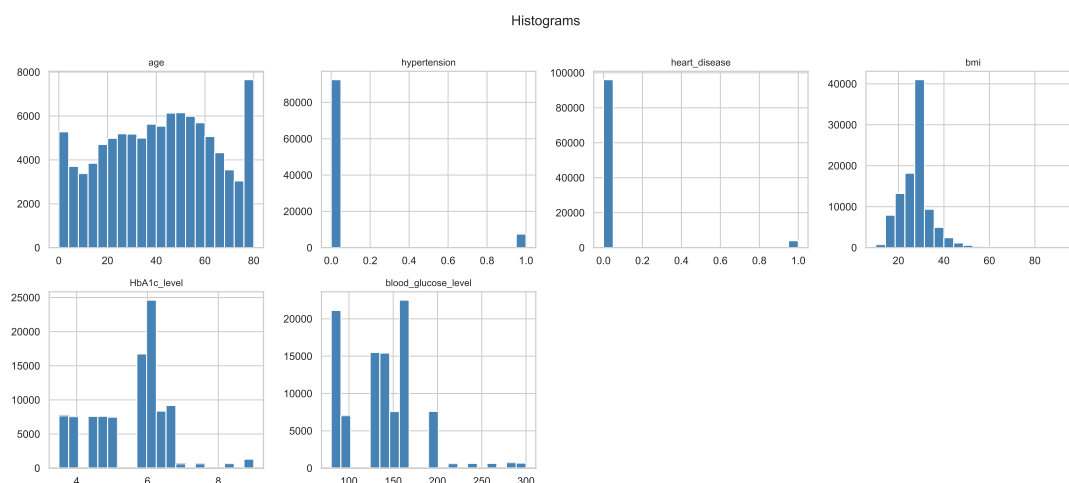


Figure 7: Diabetes - feature histograms

Inference: Age is spread fairly evenly across the adult range, while BMI looks roughly bell-shaped and centred close to the overweight threshold. HbA1c level and blood glucose level both show a long right tail, with the bulk of patients sitting in the normal range and a smaller group extending out toward diabetic values - which makes sense given these two are the clinical markers most directly tied to a diabetes diagnosis.

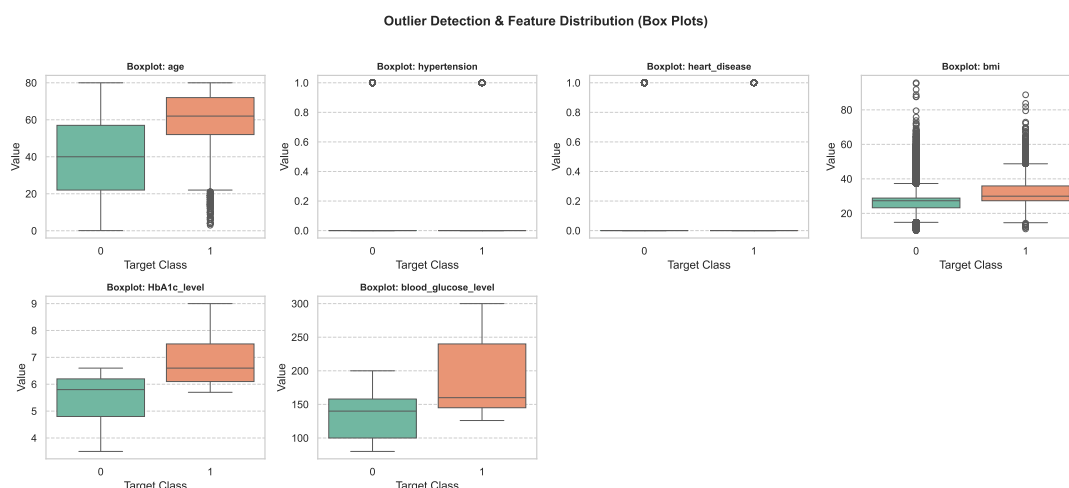


Figure 8: Diabetes - feature distributions grouped by diagnosis

Inference: Hypertension and heart disease are themselves binary flags, so their boxplots aren't especially informative on their own. The continuous features are more telling: HbA1c and glucose levels both sit visibly higher in the diabetic group, with comparatively little overlap, while age also trends a bit higher. This already suggests these will end up as the strongest predictors.

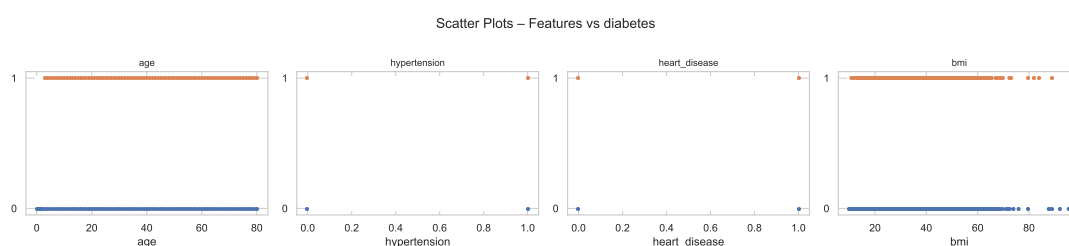


Figure 9: Diabetes - first four numeric features plotted against diagnosis

Inference: The scatter plots show the diabetic group is a much smaller, denser cluster than the non-diabetic group across most features, which is just the class imbalance showing up visually. Age and BMI don't show a sharp separation on their own - there's a lot of overlap between the two groups - reinforcing that no single feature by itself is going to carry this classification.

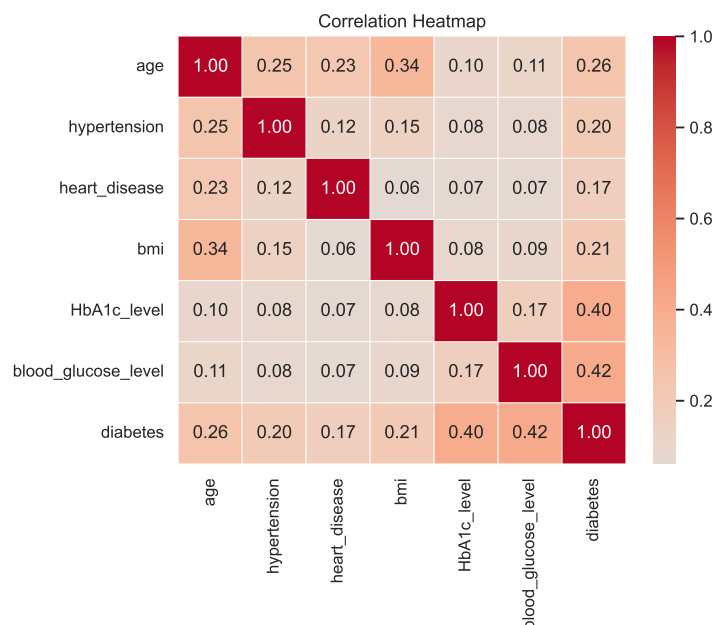


Figure 10: Diabetes - correlation heatmap of numeric features

Inference: Blood glucose level and HbA1c level show the strongest correlation with each other among the features, which tracks with both being blood-sugar-related measurements. Age shows a mild positive correlation with hypertension and heart disease, in line with these conditions becoming more common later in life. Most other feature pairs are weakly correlated, meaning there isn't much redundant information to prune before feature selection.

4.3 Loan Approval



Figure 11: Loan Approval - class distribution of the target

Inference: Approved loans outnumber rejected ones, though not by an extreme margin - this is a mild imbalance rather than the severe skew seen in the diabetes dataset. It's still worth stratifying the train/test split (which the notebook does) so both classes are represented proportionally in both sets.

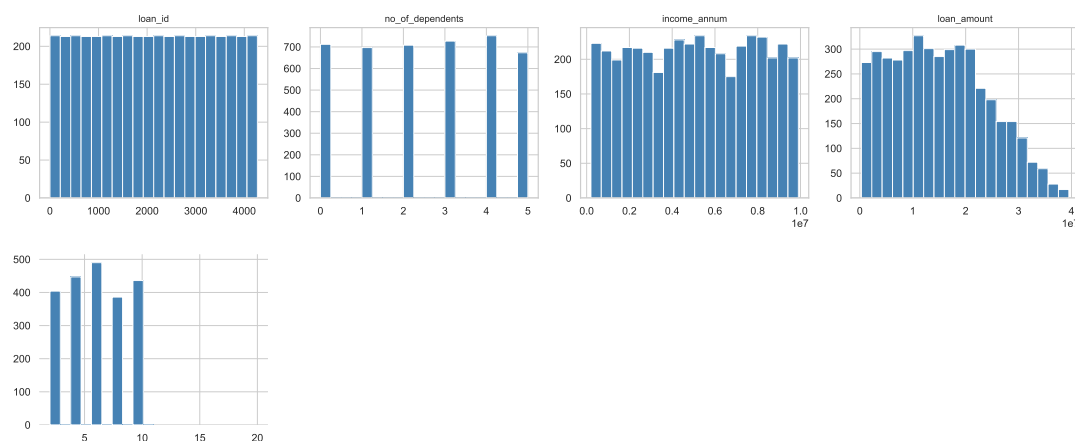


Figure 12: Loan Approval - feature histograms

Inference: Income, loan amount and the asset value columns are all right-skewed, with a long tail of high-value applicants - typical of financial data. CIBIL score, on the other hand, looks close to uniformly spread across its 300–900 range, which is a bit unusual for a credit score but works in our favour since it means the feature isn't dominated by a narrow band of values.

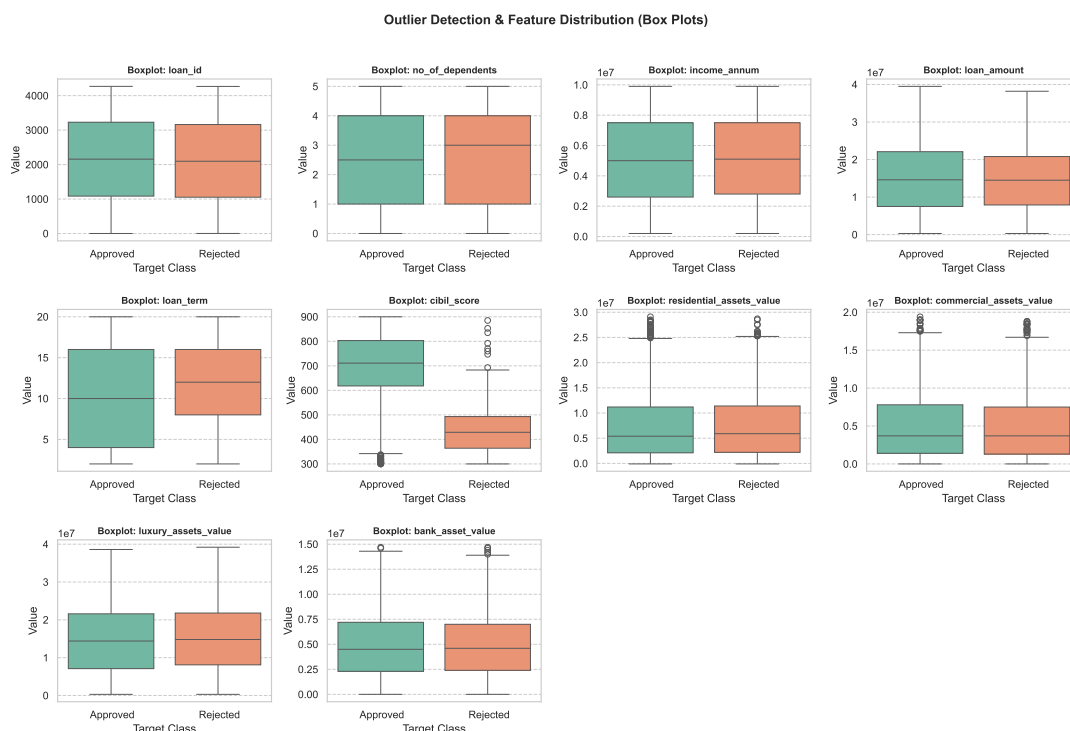


Figure 13: Loan Approval - feature distributions grouped by loan status

Inference: CIBIL score is the standout feature here - rejected applications cluster at noticeably lower scores than approved ones, with very little overlap between the two boxes. The asset and income columns show much more overlap between approved and rejected groups, suggesting credit history matters more to the approval decision than raw income or asset value in this dataset.

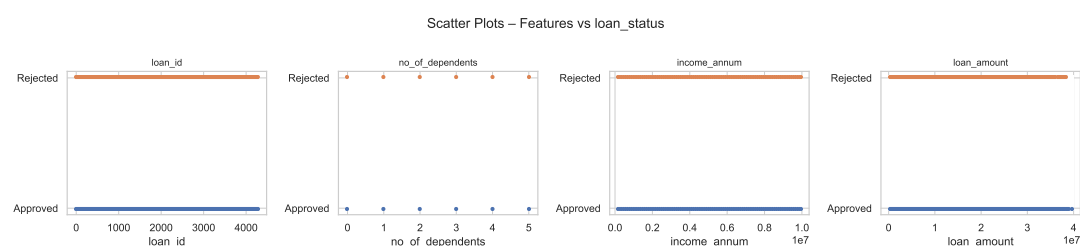


Figure 14: Loan Approval - first four numeric features plotted against loan status

Inference: Consistent with the boxplots, there isn't a clean visual split along income, loan amount or term individually - approved and rejected points are mixed throughout most of the range. This reinforces that the approval decision likely depends on some combination of features (most plausibly centred around CIBIL score) rather than any single variable in isolation.

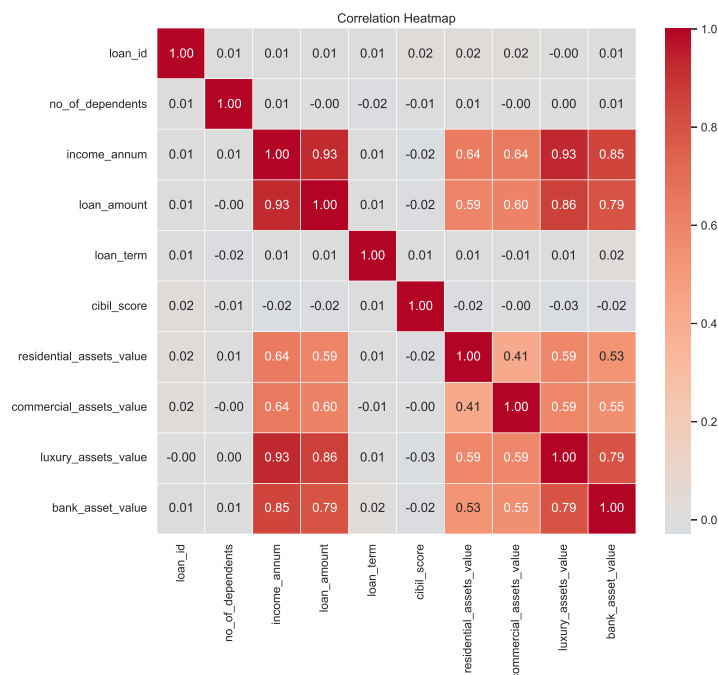


Figure 15: Loan Approval - correlation heatmap of numeric features

Inference: The four asset-value columns (residential, commercial, luxury, bank) are all strongly correlated with each other, which makes sense - wealthier applicants tend to have more of every asset type. Income and loan amount are also fairly correlated, likely because approved loan sizes tend to scale with income. This redundancy among the asset columns is worth keeping in mind, since it means feature selection can probably drop a few of them without losing much information.

4.4 MNIST Digits

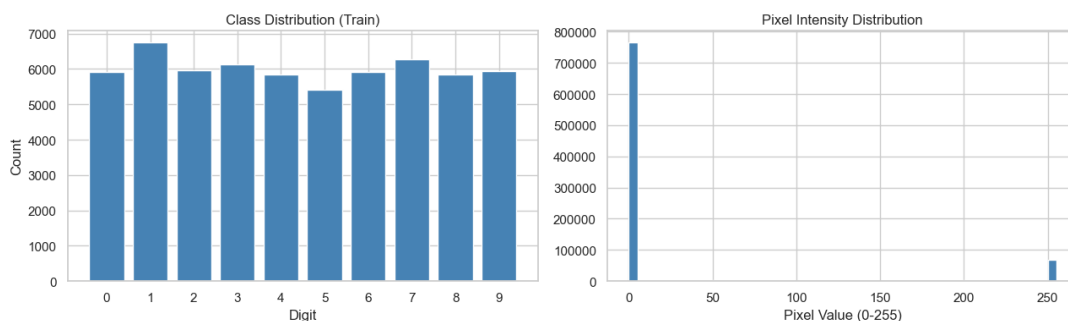


Figure 16: MNIST - class distribution (left) and pixel intensity distribution (right)

Inference: The ten digit classes are close to evenly represented in the training set, each with roughly 5,500–6,700 images, so there’s no meaningful imbalance to handle here. The pixel intensity histogram is strongly bimodal, with a tall spike near zero (background) and a smaller cluster near 255 (ink), which is exactly what you’d expect from black-background, white-stroke digit images.

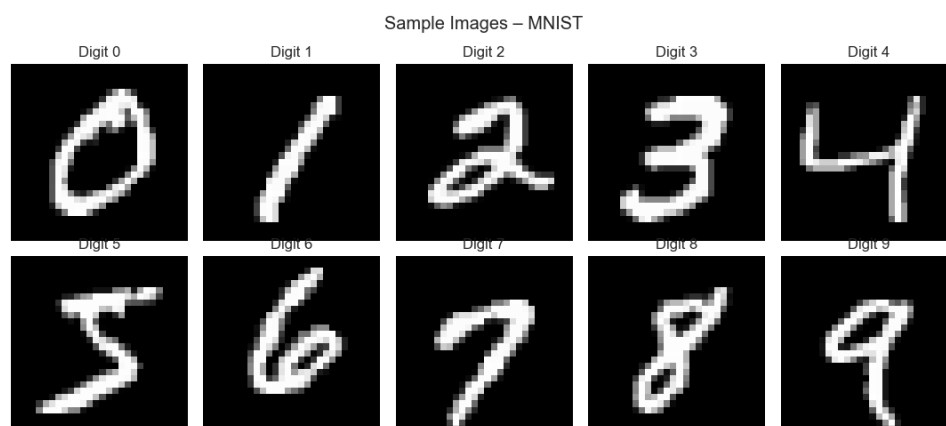


Figure 17: MNIST - one sample image per digit class

Inference: The samples confirm the images are clean, centred, and reasonably consistent in stroke width across digits. Visually, some digit pairs stand out as likely sources of confusion later on - 4 and 9, or 3 and 8, share enough curve structure that a model trained on compressed (PCA) features might mix them up more often than, say, 0 and 1.

4.5 Character Recognition (english.csv)



Figure 18: Character Recognition - class distribution across 62 classes

Inference: With 3,410 images spread over 62 classes, there are only around 55 samples per class on average, and the bars show the split isn’t perfectly even either. That’s a very small sample size per class for a model to learn

from, especially compared to MNIST's several thousand examples per digit, and it's a strong early warning sign that classification accuracy here is going to be a lot lower than on MNIST.

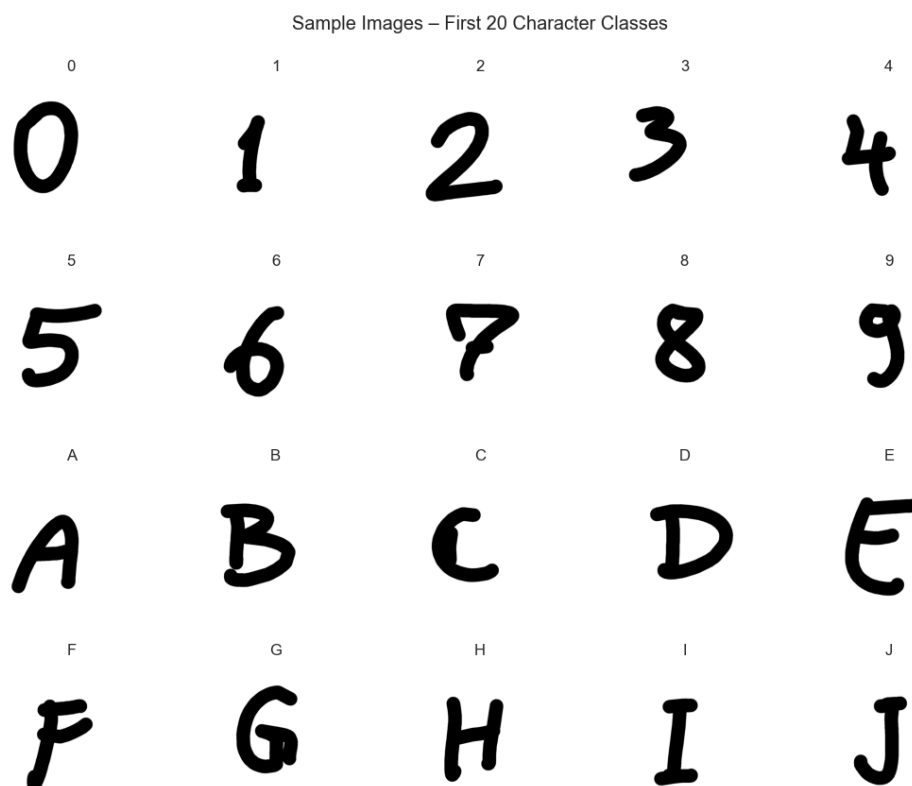


Figure 19: Character Recognition - one sample image for the first 20 classes

Inference: The character images are noisier and less standardised than MNIST - different fonts, stroke widths, and backgrounds are visible across samples. Several classes are also visually close to each other (uppercase O vs. the digit 0, lowercase l vs. digit 1), which is likely to hurt classification accuracy more than the small sample size alone would.

5 Data Preprocessing

5.1 Iris

No missing values or duplicates were found. The Id column was dropped since it carries no predictive information, and the target (Species) was label-encoded to integers. Features were scaled with `MinMaxScaler` into the $[0, 1]$ range, mainly because the ANOVA F-test used for feature selection doesn't need zero-centred data and this keeps things simple. An 80/20 train-test split was used.

5.2 Diabetes Prediction

No missing values, though 3,854 duplicate rows were flagged (kept in place for this experiment rather than removed, since the source notebook doesn't drop them). The two categorical columns, `gender` and `smoking_history`, were label-encoded. Numeric features were standardised with `StandardScaler`, and the split was stratified 80/20 to preserve the roughly 9:1 class ratio in both sets.

5.3 Loan Approval

No missing values. The `loan_id` column was dropped, and `education`, `self_employed`, and the target `loan_status` were label-encoded (approved = 0, rejected = 1). Features were standardised for the correlation/EDA stage, but for feature selection a separate `MinMaxScaler` pass was used - Chi-square selection needs non-negative inputs, and standardised data can go negative, so Min-Max scaling was applied specifically ahead of the Chi-square test. The split was stratified 80/20.

5.4 MNIST Digits

Images were loaded directly from the IDX binary files, flattened from 28×28 to 784-length vectors, and normalised to $[0, 1]$ by dividing by 255. No missing data applies here since every image in the IDX files is complete. Dimensionality was then reduced from 784 to 50 components using PCA (see Feature Selection below). The provided train/test split was kept, with 25% of the training images held out as a validation set (45,000 / 15,000 / 10,000 overall).

5.5 Character Recognition

Each image referenced in `english.csv` was opened, converted to grayscale, resized to 28×28 to match the MNIST pipeline, and flattened and normalised the same way. Labels were encoded with `LabelEncoder`. The 3,410 samples were split 60/20/20 into train, validation and test sets, stratified by class so all 62 classes appear in every split.

6 Machine Learning Algorithm

Two kinds of technique are actually doing work in this notebook: statistical feature selection (used on all four tabular/behavioural problems) and Random Forest classification (used on the two image datasets, where a model is actually trained end-to-end).

6.1 Feature Selection

ANOVA F-test (used for Iris and Diabetes) scores each feature by how much the between-class variance in that feature exceeds the within-class variance:

$$F = \frac{\text{variance between class means}}{\text{variance within classes}}$$

A high F-score means a feature's values differ a lot between classes relative to their spread within a class - exactly the kind of feature that helps a classifier. It's cheap to compute and works well for continuous features, but it only measures linear separability between class means and can miss features that matter through non-linear interactions.

Chi-square test (used for Loan Approval, after Min-Max scaling to keep values non-negative) measures how far the observed frequency of a feature's values, split by class, differs from what you'd expect if the feature and the class were independent:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

It's a natural fit for categorical or binned features but, like ANOVA, only tells you about individual feature relevance and ignores interactions between features.

PCA (used for both image datasets) doesn't score existing features at all - instead it builds a smaller set of new, uncorrelated features (principal components) that capture as much of the original variance as possible. Reducing 784 pixel values down to 50 components kept 82.46% of the variance for MNIST and 85.78% for the character set, which is a substantial compression with a fairly small variance cost, and it makes training a Random Forest on the pixel data practical.

6.2 Random Forest Classifier

Random Forest is an ensemble of decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split.

Predictions are made by majority vote across all trees. The randomness in both the data sampling and the feature subsets decorrelates the individual trees, so averaging their votes reduces variance compared to any single tree.

Advantages: handles high-dimensional data (like the 50 PCA components here) reasonably well without much tuning, is fairly robust to noisy features, and gives a usable feature-importance signal for free.

Limitations: can still overfit if trees are allowed to grow too deep or there simply isn't enough data per class, as the character recognition result later in this report illustrates; it's also slower to train and predict than simpler linear models, and less interpretable than a single decision tree.

7 Experimental Setup

Python Version

Scikit-Learn Version

IDE

Operating System

Hardware

Environment Activation `source /opt/anaconda3/bin/activate anaconda-nav`

8 Hyperparameter Selection

Parameter	Value
PCA components (MNIST & Character)	50
SelectKBest, ANOVA F-test - Iris	$k = 3$
SelectKBest, ANOVA F-test - Diabetes	$k = 5$
SelectKBest, Chi-square - Loan Approval	$k = 5$
RandomForestClassifier: <code>n_estimators</code>	100
RandomForestClassifier: <code>max_depth</code>	15
RandomForestClassifier: <code>random_state</code>	42
RandomForestClassifier: <code>n_jobs</code>	-1 (all cores)

9 Results

9.1 Iris

Preprocessing and feature selection were completed (top 3 features by ANOVA F-score: petal length, petal width, sepal length), and the data was split into

120 training and 30 test samples. No classifier was trained in this notebook for Iris - that's left for a follow-up experiment. Based on the EDA, K-Nearest Neighbours, Decision Tree, and Random Forest were flagged as suitable candidates, with accuracy and macro F1-score as the metrics to track (the dataset is balanced, so accuracy alone would already be reasonable too).

9.2 Diabetes Prediction

Feature selection settled on age, hypertension, BMI, HbA1c level, and blood glucose level as the top 5 features, with an 80,000/20,000 train-test split. As with Iris, no classifier has been trained yet. Logistic Regression, Random Forest, and SVM were identified as suitable next steps, with the notebook explicitly flagging that F1-score or ROC-AUC should be favoured over raw accuracy given the roughly 9:1 class imbalance.

9.3 Loan Approval

The top 5 features by Chi-square score were number of dependents, annual income, loan term, CIBIL score, and residential assets value, with a 3,415/854 train-test split. Model training is again deferred; Logistic Regression, Decision Tree, and Random Forest were noted as candidates, with accuracy, precision, recall, F1-score and the confusion matrix all flagged as relevant metrics.

9.4 MNIST Digits

Metric	Value
Accuracy	0.9459
Macro F1-score	0.95
Weighted F1-score	0.95

A Random Forest trained on 50 PCA components reached 94.59% test accuracy. Precision and recall stay close to 0.92–0.99 across every digit, with digits 1 and 6 coming out strongest and digits 3 and 8 slightly weaker - in line with what the sample images suggested about which digit shapes are easiest to confuse.

9.5 Character Recognition

Metric	Value
Accuracy	0.3680
Macro F1-score (first 10 classes)	0.38

The same pipeline applied to the character dataset only reached 36.80% accuracy - a large drop from MNIST. Given the class-by-class breakdown, per-class F1 scores for the first ten classes range widely, from around 0.11 up to 0.67, with no consistent pattern beyond "more distinctive-looking characters do better." This gap is discussed further below.

10 Result Visualization

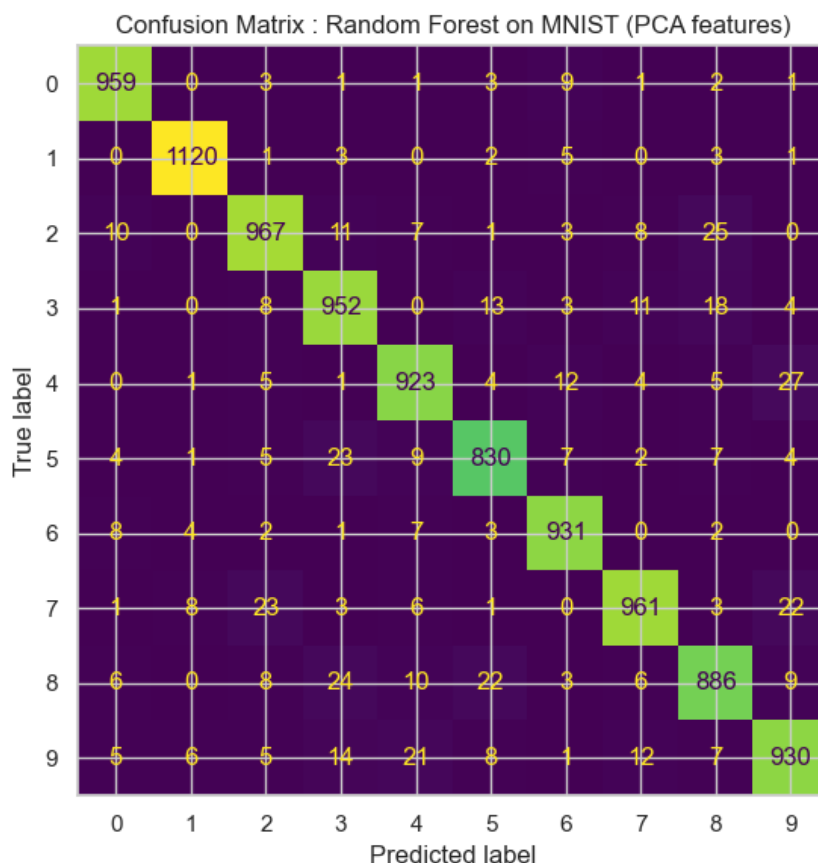


Figure 20: Confusion Matrix - Random Forest on MNIST (PCA features)

Inference: The matrix is strongly diagonal, confirming the 94.6% accuracy figure - almost every digit is classified correctly the large majority of the time. The handful of visible off-diagonal entries mostly sit near digit pairs

that share visual structure (e.g. 4/9, 3/5), which matches the confusion we anticipated from the sample images earlier.

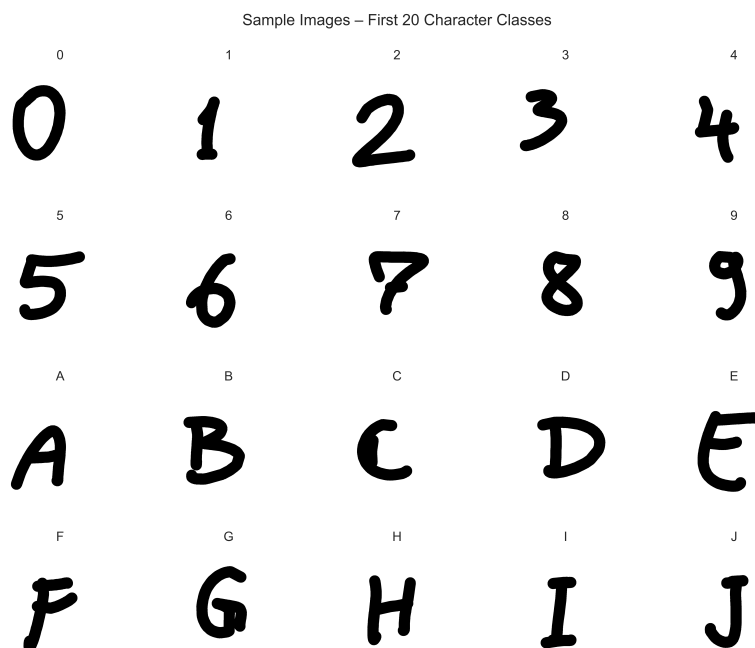


Figure 21: Confusion Matrix - Random Forest on Character Recognition

Inference: Compared to the MNIST matrix, this one is visibly noisier, with far more mass scattered off the diagonal, consistent with the much lower 36.8% accuracy. With roughly 55 training examples per class spread across 62 classes, the model simply doesn't see enough examples of most characters to learn a reliable boundary between visually similar ones, which is the most likely explanation for the spread seen here.

11 Discussion

The four problems in this experiment sit at quite different points on the difficulty spectrum, and the EDA mostly explains why. Iris is close to a solved problem before any model is even trained - the petal measurements alone nearly separate the three species - while Diabetes and Loan Approval both carry class imbalance that will need careful metric choices (and possibly re-sampling) once a classifier is actually trained on them.

The choice of feature-selection method also had to be adapted per dataset rather than applied uniformly: Chi-square needed non-negative inputs, which ruled it out for the standardised diabetes features (hence ANOVA there) but worked fine for loan data once it was Min-Max scaled separately. This kind

of dataset-specific adjustment is easy to miss if the pipeline is copy-pasted without checking each dataset's scaling assumptions.

The clearest result in this experiment is the gap between MNIST (94.6% accuracy) and the custom character set (36.8%), using the exact same pipeline and the same Random Forest hyperparameters on both. The difference isn't the algorithm - it's the data: MNIST gives the model thousands of examples per class, the character set gives it barely fifty. That's a useful reminder that a PCA + Random Forest pipeline that looks solid on a benchmark dataset can fall apart quickly once the amount of data per class drops.

There are a few clear limitations worth flagging. No hyperparameter tuning or cross-validation was done anywhere in this notebook - a single train-test split and a fixed set of Random Forest parameters were used throughout, so the reported numbers should be read as a baseline rather than a ceiling. Three of the five problems (Iris, Diabetes, Loan) stop at the preprocessing stage with no trained model at all yet. For the character dataset specifically, class imbalance across the 62 categories wasn't addressed, and no data augmentation was attempted despite the small sample count being the likely main bottleneck.

12 Conclusion

Across five datasets, this experiment worked through exploratory analysis, cleaning, encoding, scaling, and feature selection, tailoring each step to what the data actually needed rather than applying one fixed recipe. Where a model was actually trained - the two image datasets - the same Random Forest and PCA setup produced very different outcomes purely as a function of how much data was available per class, 94.6% accuracy on MNIST against 36.8% on the much smaller character set. The main open items for a follow-up experiment are training and evaluating classifiers on Iris, Diabetes, and Loan Approval (all preprocessing-ready), and investigating whether more data, augmentation, or a different model narrows the gap seen on the character recognition task.